

```

..In getCombos('C', 1) results are ['C']
.Adding head 'B' to tail combos:
.New combination BC
.In getCombos('BC', 2) part 2, get combos without head 'B')
..In getCombos('C', 2), start.
..In getCombos('C', 2) part 1, get combos with head 'C'
...In getCombos('', 1), start.
...In getCombos('', 1) base case returns []
..Adding head 'C' to tail combos:
..In getCombos('C', 2) part 2, get combos without head 'C')
...In getCombos('', 2), start.
...In getCombos('', 2) base case returns []
..In getCombos('C', 2) results are []
.In getCombos('BC', 2) results are ['BC']
In getCombos('ABC', 2) results are ['AB', 'AC', 'BC']
Results: ['AB', 'AC', 'BC']

```

Every `getCombos()` function call has two recursive calls for the two parts of the algorithm. For our `getCombos('ABC', 2)` example, the first part ❶ is to get all the combinations that include the head A. In the tree, this generates all the combinations *under* the A node in the 1-combination level.

We can do this by passing the tail and $k - 1$ to the first recursive function call: `getCombos('BC', 1)` ❷. We add A to each combination that this recursive function call returns. Let's use the leap-of-faith principle and just assume our `getCombos()` correctly returns a list of k -combinations, `['B', 'C']`, even though we haven't finished writing it yet. We now have all the k -combinations that include the head A in an array to hold our results: `['AB', 'AC']`.

The second part ❸ gets all the combinations that don't include the head A. In the tree, this generates all the combinations *to the right* of the A node in the 1-combination level. We can do this by passing the tail and k to the second recursive function call: `getCombos('BC', 2)`. This returns `['BC']`, since BC is the only 2-combination of BC.

The results from `getCombos('ABC', 2)`'s two recursive calls, `['AB', 'AC']` and `['BC']`, are concatenated together and returned: `['AB', 'AC', 'BC']` ❹. The `getCombos()` function generates combinations of larger sets the same way.